

IEORE4004

Optimization Models and Methods

Yaren Bilge Kaya, PhD

Columbia University
Department of Industrial Engineering and Operations Research



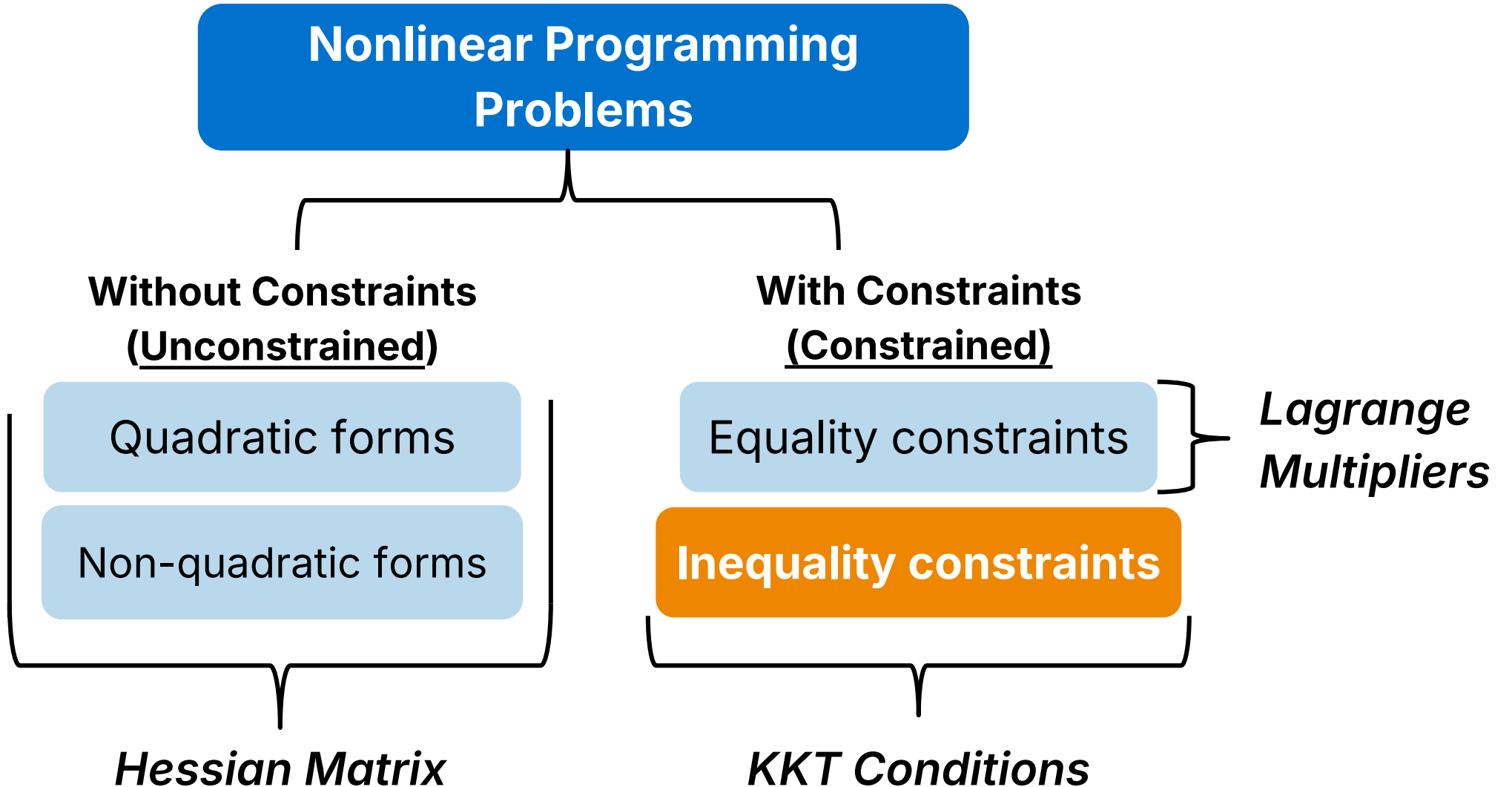
Outline

- Optimality Condition: Inequality Case
 - KKT Conditions
 - KKT Example
- Algorithmically Solving Nonlinear Optimization Problems
 - Gradient Descent Method
 - Newton's Method

Optimality Condition: Inequality Case

- KKT Conditions
- KKT Example

Types of NLPs



Optimality Conditions: Inequality Case

Today we consider the more general **inequality-constrained NLP**:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{subject to} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_\ell(x) = 0, \quad \ell = 1, \dots, p. \end{aligned}$$

- Similar to the **equality** constrained case, but with new **inequalities** present
- The corresponding Lagrangian function is simply:

$$L(x, \lambda, \nu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{\ell=1}^p \nu_\ell h_\ell(x)$$

- The Lagrange multiplier associated with each $g_i(x) \leq 0 : \lambda_i$
- The Lagrange multiplier associated with each $h_\ell(x) = 0 : \nu_\ell$

A Bit of History: Kuhn and Tucker

- Kuhn and Tucker: mathematicians at Princeton
 - Kuhn and John Nash were contemporaries as graduate students
 - Tucker was Nash's advisor
- Kuhn and Tucker in 1951 published the key *necessary optimality conditions* for nonlinear optimization problems
 - Known as the **Kuhn-Tucker (KT) conditions**

A Bit of History: Kuhn and Tucker

- Kuhn and Tucker: mathematicians at Princeton
 - Kuhn and John Nash were contemporaries as graduate students
 - Tucker was Nash's advisor
- Kuhn and Tucker in 1951 published the key **necessary optimality conditions** for nonlinear optimization problems
 - Known as the **Kuhn-Tucker (KT) conditions**
- Only problem was, they were actually discovered earlier, in 1939, by William Karush!
 - In his MS Thesis from the University of Chicago
 - Never published because he didn't think the results were that important!

KKT Conditions

Karush-Kuhn-Tucker (KKT) Conditions

Represent *necessary optimality conditions* for any solution x to *inequality*-constrained nonlinear optimization problems:

$$1. \quad g_i(x) \leq 0, i = 1, \dots, m; \quad h_\ell(x) = 0, \ell = 1, \dots, p \quad \mathbf{1) \textit{Primal feasible}}$$

The solution must satisfy all the constraints of the problem.

Karush-Kuhn-Tucker (KKT) Conditions

Represent *necessary optimality conditions* for any solution x to *inequality*-constrained nonlinear optimization problems:

1. $g_i(x) \leq 0, i = 1, \dots, m; h_\ell(x) = 0, \ell = 1, \dots, p$ **1) Primal feasible**
2. $\lambda_i \geq 0, i = 1, \dots, m$ **2) Sign restrictions**

Karush-Kuhn-Tucker (KKT) Conditions

Represent *necessary optimality conditions* for any solution x to *inequality*-constrained nonlinear optimization problems:

- | | |
|--|-----------------------------------|
| 1. $g_i(x) \leq 0, i = 1, \dots, m; h_\ell(x) = 0, \ell = 1, \dots, p$ | 1) Primal feasible |
| 2. $\lambda_i \geq 0, i = 1, \dots, m$ | 2) Sign restrictions |
| 3. $\lambda_i g_i(x) = 0, i = 1, \dots, m$ | 3) Complementary Slackness |

For each inequality constraint, either the constraint is 'active' (i.e., it holds as an equality at the optimal point), or its corresponding Lagrange multiplier is zero

Karush-Kuhn-Tucker (KKT) Conditions

Represent *necessary optimality conditions* for any solution x to *inequality*-constrained nonlinear optimization problems:

- | | |
|--|-----------------------------------|
| 1. $g_i(x) \leq 0, i = 1, \dots, m; h_\ell(x) = 0, \ell = 1, \dots, p$ | 1) Primal feasible |
| 2. $\lambda_i \geq 0, i = 1, \dots, m$ | 2) Sign restrictions |
| 3. $\lambda_i g_i(x) = 0, i = 1, \dots, m$ | 3) Complementary Slackness |

For each inequality constraint, either the constraint is 'active' (i.e., it holds as an equality at the optimal point), or its corresponding Lagrange multiplier is zero

If λ_i is the multiplier for the i -th constraint $g_i(x) \leq 0$, then $\lambda_i g_i(x) = 0$ at the optimal solution.

This means that if $g_i(x) < 0$ (constraint not 'active'), then $\lambda_i = 0$, and if $\lambda_i > 0$, then $g_i(x) = 0$ (constraint is 'active')

Karush-Kuhn-Tucker (KKT) Conditions

Represent *necessary optimality conditions* for any solution x to *inequality*-constrained nonlinear optimization problems:

- | | |
|---|-----------------------------------|
| 1. $g_i(x) \leq 0, i = 1, \dots, m; \quad h_\ell(x) = 0, \ell = 1, \dots, p$ | 1) Primal feasible |
| 2. $\lambda_i \geq 0, i = 1, \dots, m$ | 2) Sign restrictions |
| 3. $\lambda_i g_i(x) = 0, i = 1, \dots, m$ | 3) Complementary Slackness |
| 4. $\nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) + \sum_{\ell=1}^p \nu_\ell \nabla h_\ell(x) = 0$ | 4) Stationarity |

? What does the last condition represent?

Karush-Kuhn-Tucker (KKT) Conditions

Represent *necessary optimality conditions* for any solution x to *inequality*-constrained nonlinear optimization problems:

- | | |
|---|-----------------------------------|
| 1. $g_i(x) \leq 0, i = 1, \dots, m; h_\ell(x) = 0, \ell = 1, \dots, p$ | 1) Primal feasible |
| 2. $\lambda_i \geq 0, i = 1, \dots, m$ | 2) Sign restrictions |
| 3. $\lambda_i g_i(x) = 0, i = 1, \dots, m$ | 3) Complementary Slackness |
| 4. $\nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) + \sum_{\ell=1}^p \nu_\ell \nabla h_\ell(x) = 0$ | 4) Stationarity |

? What does the last condition represent?

The last condition states that the gradient of the objective can be represented as a linear combination of the constraint gradients

A Note on Sign Restrictions

The sign restrictions on λ_i vary based on a **minimization** or **maximization** problem

The interpretation of λ_i is: *what happens to the objective function when the right-hand side of inequality constraint i increases*

There are four cases, depending on **objective function direction** and **sense of the inequality constraint**:

1. $\min f(x) : g_i(x) \leq 0 \Rightarrow \lambda_i \geq 0$

2. $\min f(x) : g_i(x) \geq 0 \Rightarrow \lambda_i \leq 0$

3. $\max f(x) : g_i(x) \geq 0 \Rightarrow \lambda_i \geq 0$

4. $\max f(x) : g_i(x) \leq 0 \Rightarrow \lambda_i \leq 0$

KKT Examples

Practice: Solving Using KKT

Consider the following optimization problem:

$$\begin{array}{ll}\max & 3x_1 + x_2 \\ & x_1, x_2 \in \mathbb{R} \\ \text{subject to} & x_1^2 + x_2^2 \leq 5 \\ & x_1 - x_2 \leq 1\end{array}$$

Practice: Solving Using KKT

Consider the following optimization problem:

$$\begin{aligned} & \max_{x_1, x_2 \in \mathbb{R}} && 3x_1 + x_2 \\ & \text{subject to} && x_1^2 + x_2^2 \leq 5 \\ & && x_1 - x_2 \leq 1 \end{aligned}$$

- Any (local) maxima must satisfy the four **KKT conditions**:

$$\nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) + \sum_{\ell=1}^p \nu_\ell \nabla h_\ell(x) = 0$$

$$\lambda_i g_i(x) = 0, i = 1, \dots, m$$

$$g_i(x) \leq 0, i = 1, \dots, m; \quad h_\ell(x) = 0, \ell = 1, \dots, p$$

$$\lambda_i \leq 0, i = 1, \dots, m$$

Practice: Solving Using KKT

Lagrangian Function:

$$L(x, \lambda) = 3x_1 + x_2 + \lambda_1(x_1^2 + x_2^2 - 5) + \lambda_2(x_1 - x_2 - 1)$$

Complementary Slackness:

$$(A) \quad \lambda_1(x_1^2 + x_2^2 - 5) = 0$$
$$(B) \quad \lambda_2(x_1 - x_2 - 1) = 0$$

Stationary:

$$(C) \quad \frac{\partial L}{\partial x_1} = 3 + 2\lambda_1 x_1 + \lambda_2 = 0$$
$$(D) \quad \frac{\partial L}{\partial x_2} = 1 + 2\lambda_1 x_2 - \lambda_2 = 0$$

Practice: Solving Using KKT

Complementary Slackness:

$$(A) \quad \lambda_1(x_1^2 + x_2^2 - 5) = 0$$
$$(B) \quad \lambda_2(x_1 - x_2 - 1) = 0$$

Stationary:

$$(C) \quad \frac{\partial L}{\partial x_1} = 3 + 2\lambda_1 x_1 + \lambda_2 = 0$$
$$(D) \quad \frac{\partial L}{\partial x_2} = 1 + 2\lambda_1 x_2 - \lambda_2 = 0$$

When we look at (A) $\rightarrow$ either $\lambda_1=0$, or $x_1^2+x_2^2 = 5$ or both.

If $\lambda_1=0$, then in (C) $\lambda_2 = -3$ and in (D) $\lambda_2 = 1$.

Therefore, we say $\lambda_1 \neq 0$ and $x_1^2+x_2^2 = 5$ (E)

Practice: Solving Using KKT

Complementary Slackness: (A) $\lambda_1(x_1^2 + x_2^2 - 5) = 0$

$$(B) \quad \lambda_2(x_1 - x_2 - 1) = 0$$

Stationary:

$$(C) \quad \frac{\partial L}{\partial x_1} = 3 + 2\lambda_1 x_1 + \lambda_2 = 0$$

$$(D) \quad \frac{\partial L}{\partial x_2} = 1 + 2\lambda_1 x_2 - \lambda_2 = 0$$

When we look at (B) $\rightarrow$ either $\lambda_2 = 0$ or $x_1 - x_2 = 1$ or both.

If $\lambda_2 = 0$, then in (C) $x_1 = \frac{-3}{2\lambda_1}$ and in (D) $x_2 = \frac{-1}{2\lambda_1}$.

From (E) $x_1^2 + x_2^2 = 5$, we have $\left(\frac{-3}{2\lambda_1}\right)^2 + \left(\frac{-1}{2\lambda_1}\right)^2 = 5 \Rightarrow \frac{10}{4\lambda_1^2} = 5 \Rightarrow \lambda_1^2 = \frac{1}{2}$

Practice: Solving Using KKT

We know from this optimization problem being a maximization problem with $\leq$ inequality, we'll have $\lambda \leq 0$ then $\lambda_1 = -\frac{\sqrt{2}}{2}$

From (C), plug in $\lambda_1 = -\frac{\sqrt{2}}{2}$ and $\lambda_2 = 0$, we got $x_1 = \frac{-3}{2\lambda_1} = \frac{3\sqrt{2}}{2}$

Similarly, from (D), we got $x_2 = \frac{-1}{2\lambda_1} = \frac{\sqrt{2}}{2}$

But then: $x_1 - x_2 = \frac{3\sqrt{2}}{2} - \frac{\sqrt{2}}{2} = \sqrt{2} \not\leq 1$ (constraint 2 violated!)

Thus, we can say that $\lambda_2 \neq 0$ and $x_1 - x_2 = 1$ (F).

Practice: Solving Using KKT

From (E) $x_1^2 + x_2^2 = 5$ and (F) $x_1 - x_2 = 1$, we can solve for x_1 and x_2 .

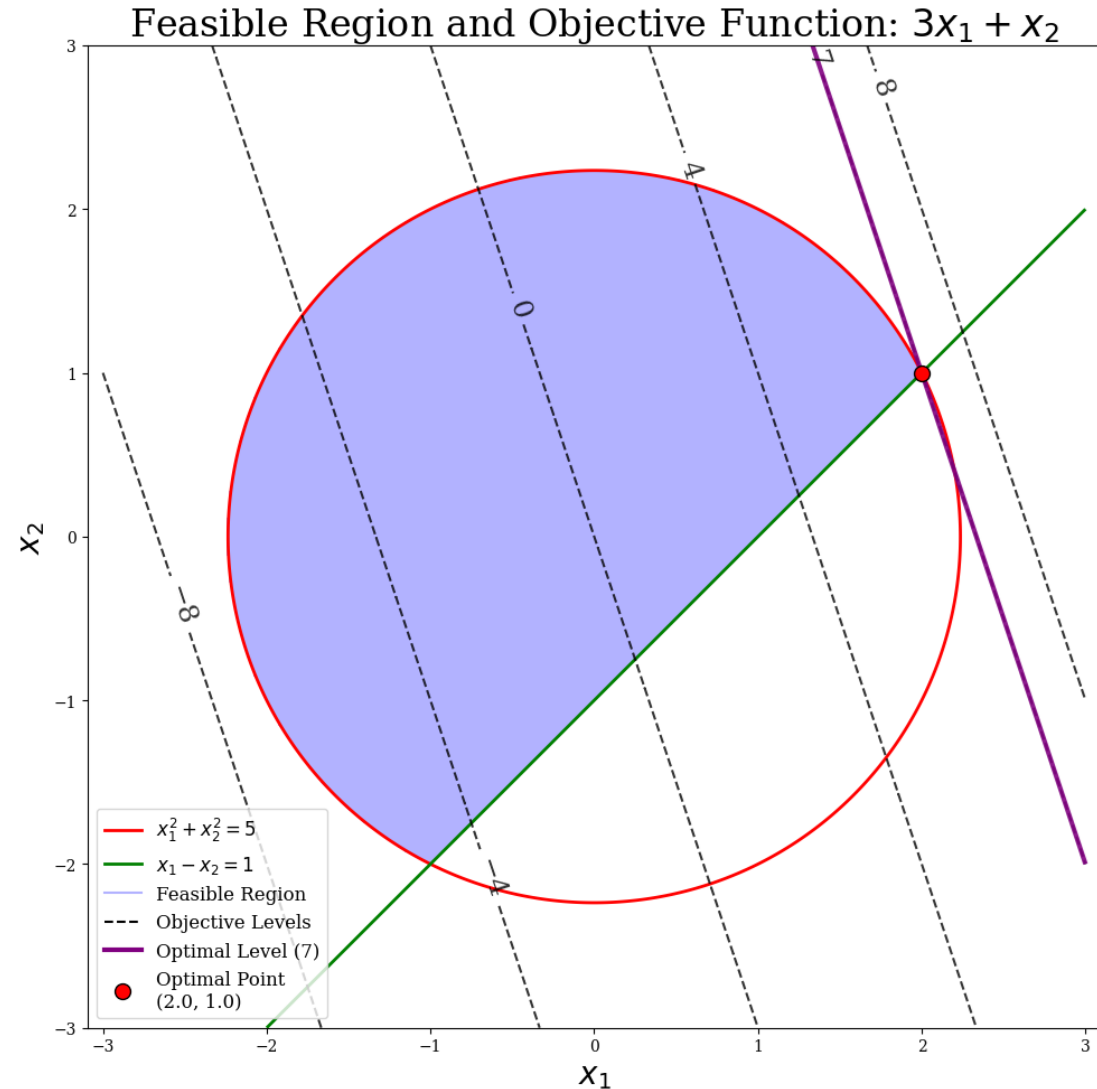
The solutions are $(-1, -2)$ and $(2, 1)$ – verify these by yourself!

- Solution $(-1, -2)$ and equation (C) gives us $\lambda_1 = 2/3$ (violates sign restriction)
- Solution $(2, 1)$ and equation (C) and (D) gives us $\lambda_1 = -2/3$ and $\lambda_2 = -1/3$

To summarize: the optimal solution is $(x_1, x_2, \lambda_1, \lambda_2) = (2, 1, -2/3, -1/3)$

Practice: Graphical Solution

$$\begin{array}{ll}\max & 3x_1 + x_2 \\ \text{subject to} & x_1, x_2 \in \mathbb{R} \\ & x_1^2 + x_2^2 \leq 5 \\ & x_1 - x_2 \leq 1\end{array}$$



Algorithmically Solving Nonlinear Optimization Problems

- Gradient Descent Method
- Newton's Method

Unconstrained Optimization

Consider a general **unconstrained** nonlinear optimization problem with convex, continuous, and twice differentiable objective function and convex feasible region: $\min_{x \in \mathbb{R}^n} f(x)$

- In a few special cases, we can find a solution **analytically** using the necessary optimality condition: $\nabla f = 0$

Unconstrained Optimization

Consider a general **unconstrained** nonlinear optimization problem with convex, continuous, and twice differentiable objective function and convex feasible region:

$$\min_{x \in \mathbb{R}^n} f(x)$$

- In a few special cases, we can find a solution ***analytically*** using the necessary optimality condition: $\nabla f = 0$
- Typically, though, such problems must still be solved by an iterative algorithm...

***Most unconstrained optimization problems
have no closed form solution***

How Might One Construct a Method?

- Suppose we know some point x^0 not too far away from minimum
- What should we do?

General Descent Method

Starting from an initial point x^0 , we want to move in an improving direction Δx

- How long of a movement? Take a step of length $t \Delta x$ that's:
 - Not too short
 - Not too long

General Descent Method

Starting from an initial point x^0 , we want to move in an improving direction Δx

- When should we stop moving?
 - When there is (approximately) no more improving direction!
 - That is, when $\nabla f \approx 0$
 - This will lead to a stationary point
 - If the problem is a convex optimization problem, then also a **global** minimum

General Descent Method

Algorithm 1 General Descent Method

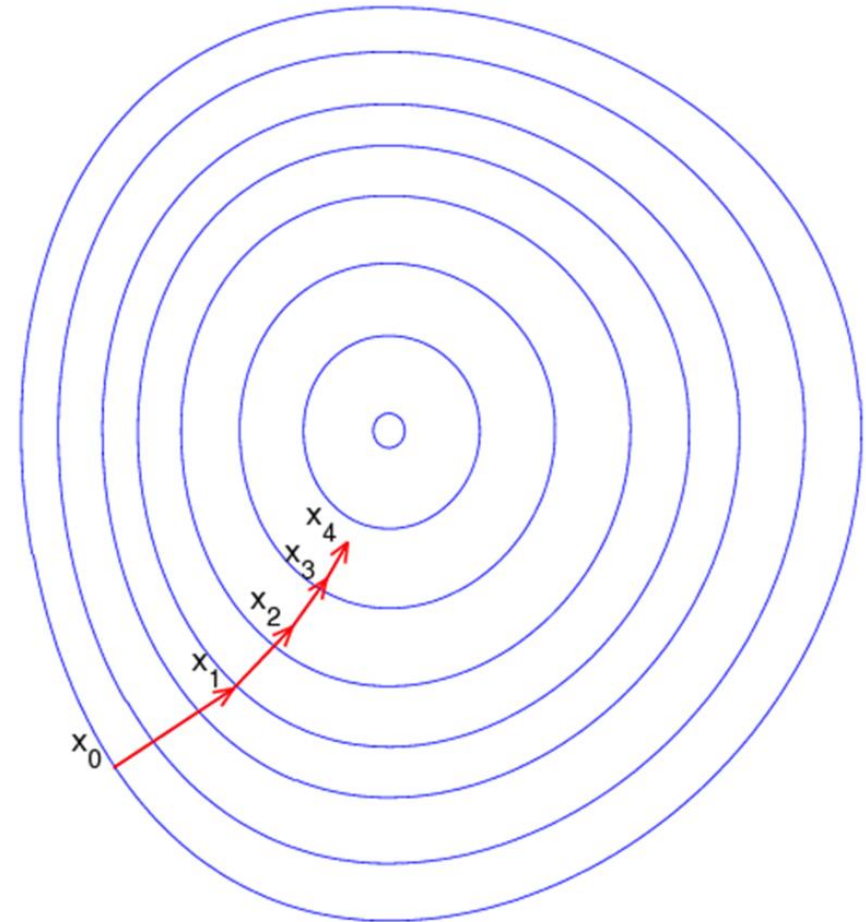
Input: Convex, twice continuously differentiable function $f(x)$, and starting point x^0 .

Output: Optimal solution x^k .

- 1: Let $k \leftarrow 0$.
 - 2: **repeat**
 - 3: Determine descent direction Δx^k .
 - 4: *Line search.* Choose a step size $t^k > 0$.
 - 5: *Update.* $x^{k+1} \leftarrow x^k + t^k \Delta x^k$, and $k \leftarrow k + 1$.
 - 6: **until** *Stopping condition satisfied.*
-

Important parameters:

- Starting point x^0
- Descent direction Δx
- Step size t



http://commons.wikimedia.org/wiki/File:Gradient_descent.png

Gradient Descent Method

A Note about the Gradient

- The gradient ∇f for a function $f(x)$, at point x , represents the ***direction of greatest increase*** at x
- What is the gradient ∇f for simple function $f(x_1, x_2) = x_1 + x_2$?

A Note about the Gradient

- The gradient ∇f for a function $f(x)$, at point x , represents the *direction of greatest increase* at x
- What is the gradient ∇f for simple function $f(x_1, x_2) = x_1 + x_2$?
- It is just: $\nabla f = [1, 1]$ (this is the *direction of greatest increase*)
- Thus, for minimization, $-\nabla f$ gives the **best improving direction**

More on Steepest Descent Method

We can use $-\nabla f$, the negative of the gradient, for the **best** improving direction

- This is known as the **gradient descent method**
- However, it only improves so far, before it needs updated
 - The gradient at point x^{k+1} is not likely the same as at point x^k
- The step size t can be chosen using one of several **line search** approaches

Steepest Descent: Good and Bad

Good: Relatively simple and straightforward method

Good: Not too many parameters

Good: Performs quite well at first

- That is, good improvement in initial iterations

Bad: It is *famous (infamous?)* for *very slow* convergence

- That is, as it approaches the optimal solution, many iterations are required until the stopping condition is satisfied

Steepest Descent via Python

- The Python script "SteepestDescent.ipynb" uses steepest descent to find the global minimum of the function:

$$f(x) = x_1^2 + x_1x_2 + 3x_2^2$$

- The script implements an iterative algorithm to find the global minimum (within tolerance) of $f(x)$

- Original due to ***Dr. James Allison at the University of Illinois***

- SteepestDescent.ipynb is on course website

Steepest Descent via Python

- The Python script "SteepestDescent.ipynb" uses steepest descent to find the global minimum of the function:

$$f(x) = x_1^2 + x_1x_2 + 3x_2^2$$

- The script implements an iterative algorithm to find the global minimum (within tolerance) of $f(x)$

- Original due to ***Dr. James Allison at the University of Illinois***

- SteepestDescent.ipynb is on course website

- Notice what happens when this script is run:

- It starts from an initial point $x^0 = [3, 3]^T$; what's used for t ? Δx ?

- How many iterations to converge? What if x^0 is varied?

Using Additional Information

- The Steepest Descent Method uses only gradient ∇f information

Using Additional Information

- The Steepest Descent Method uses only gradient ∇f information
- More information is available from the Hessian H , which provides information on the *curvature* of the function $f(x)$

Using Additional Information

- The Steepest Descent Method uses only gradient ∇f information
- More information is available from the Hessian H , which provides information on the **curvature** of the function $f(x)$
- The step direction $-H^{-1}\nabla f$ (rather than just the negative of the gradient $-\nabla f$) is called the **deflected gradient**
 - It has a fixed step size $t = 1$
 - Incorporates second-derivative information

Newton's Method

Newton's Method

The following algorithm is a generic version of Newton's Method

Algorithm 2 Newton's Method

Input: Convex, twice continuously differentiable function $f(x)$, and starting point x^0 .

Output: Optimal solution x^k .

- 1: Let $k \leftarrow 0$, and set $t \leftarrow 1$.
 - 2: **repeat**
 - 3: Set $\Delta x^k \leftarrow -H^{-1}(x^k)\nabla f(x^k)$.
 - 4: Update. $x^{k+1} \leftarrow x^k + t\Delta x^k$, and $k \leftarrow k + 1$.
 - 5: **until** *Stopping condition satisfied*.
-

Newton's Method

A lot depends on the initial point x^0 :

- If “close enough” to the minimum, method has ***rapid convergence***
- However, if initial point is too far away, ***can fail to converge!***
- Often, a combination of gradient descent and Newton's Method approaches can work well – there are ***many variants*** of these approaches

The Python script "DeflectedGradDescent.ipynb" uses Newton's method to find the global minimum of the function:

$$f(x) = x_1^2 + x_1x_2 + 3x_2^2$$

Initialization Methods

Initialization Methods | Random Initialization

- **What it is:** Choose a point randomly from within the feasible region of the problem.
- **How it works:**
 - A uniform or Gaussian distribution can be used to sample the point, depending on the nature of the feasible region.
 - In constrained optimization, random points must respect the constraints.
- **Advantages:**
 - Useful when no prior knowledge about the problem is available.
 - In nonconvex problems, multiple random initializations can help explore different basins of attraction.
- **Disadvantages:**
 - May lead to poor convergence or slow progress if the chosen point is far from the optimal region.

Initialization Methods | Grid Search-Based Initialization

- **What it is:** Choose an initial point from a predefined grid over the search space.
- **How it works:**
 - Divide the search space into a grid and test points systematically.
 - Often used in conjunction with coarse-to-fine strategies: Start with a coarse grid and refine around promising areas.
- **Advantages:**
 - Ensures coverage of the search space.
 - Useful for small-dimensional problems.
- **Disadvantages:**
 - Computationally expensive in higher dimensions.

Initialization Methods | Warm-Start Initialization

- **What it is:** Use a solution from a similar or related problem as the initial point.
- **How it works:**
 - For example, if you're solving a sequence of optimization problems (e.g., parameter tuning), use the solution of the previous problem.
- **Advantages:**
 - Can leverage continuity or similarity between problems for faster convergence.
- **Disadvantages:**
 - May not generalize well if the current problem differs significantly from past ones.

Initialization Methods | Clustering-Based Initialization

- **What it is:** Use clustering techniques to identify candidate regions in the search space and select a point from one of these clusters.
- **How it works:**
 - Use k-means or hierarchical clustering on sampled data points or previous solutions.
 - Pick the centroid of a cluster as the initial point.
- **Advantages:**
 - Helps identify diverse starting points in multimodal problems.
- **Disadvantages:**
 - Additional computational effort is required for clustering.

Initialization Methods | Heuristic or Informed

- **What it is:** Use problem-specific knowledge to pick an initial point.
- **How it works:**
 - Analyze the problem context or use domain expertise to identify a good starting region.
 - For instance, if you are optimizing a function in physics, you might start with physically meaningful parameters.
- **Advantages:**
 - Can significantly reduce convergence time.
- **Disadvantages:**
 - Requires prior knowledge or insight, which may not always be available.

Reminder

Gurobi can handle:

- **Convex NLPs:** These are nonlinear programs where the objective and constraint functions are convex. Convex problems are easier to solve because any local minimum is also a global minimum.
- **Quadratically Constrained Quadratic Programs (QCQP):** These involve an objective function and constraints that are both quadratic.
- **Quadratic Programs (QP):** A special case of QCQP, where the objective function is quadratic, and the constraints are linear.

NonConvex Functions in Gurobi

```
m.setParam('NonConvex', a)
```

a=0: This is the default value. Gurobi assumes that the quadratic model is convex and will return an error if it detects non-convexity.

a=1: Gurobi will allow the optimization of non-convex quadratic models. However, it will not necessarily find a global optimum; it might find a local optimum instead.

a=2: Setting NonConvex to 2 enables Gurobi to solve non-convex quadratic models to global optimality. This means Gurobi will use more sophisticated and computationally intensive methods to explore the solution space thoroughly and find the global optimum, rather than just a local optimum.

More on Nonlinear Programming

For those of you interested in learning more about nonlinear programming, I recommend the following books:

- "Numerical Optimization" by Jorge Nocedal and Stephen Wright
- "Nonlinear Programming" by Dimitri Bertsekas
- You can also take a look at the [NLP Notes](#) I have included on Courseworks.